Implementation of selected quantum algebra and number theory algorithms

Mateusz Hurkała

Master thesis
Tutor: D.Sc. Krzysztof Turowski

Jagiellonian University
Department of Mathematic and Computer Science
Kraków 2026

Contents

1 Theoretical Foundations of Quantum Computation **2**
1.1 Introduction . . . 2
1.2 Mathematical Prerequisites . . . 3
1.2.1 Hilbert Spaces and Dirac Notation . . . 3
1.2.2 Inner and Outer Products . . . 4
1.2.3 Linear Operators . . . 5
1.3 The Qubit . . . 5
1.3.1 Computational Basis . . . 6
1.3.2 Superposition . . . 6
1.3.3 The Bloch Sphere . . . 6
1.4 Composite Systems and Tensor Products . . . 8
1.5 Quantum Entanglement . . . 9
1.5.1 The Bell States . . . 9
1.6 Quantum Logic Gates and Circuits . . . 10
1.6.1 Single-Qubit Gates . . . 10
1.6.2 Two Qubit Gates: CNOT, SWAP, and Controlled-Phase . . . 11
1.6.3 Advanced and Multi-Controlled Gates . . . 13
1.7 Universal Quantum Computation . . . 13
1.8 Theoretical Machine and Memory Models . . . 14
1.8.1 The Quantum Turing Machine (QTM) . . . 14
1.8.2 Equivalence of QTMs and Quantum Circuits . . . 15
1.8.3 Quantum Random Access Memory (QRAM) . . . 16
1.9 Computational Complexity and Quantum Advantage . . . 17
1.9.1 Definitions of Complexity Classes . . . 17
1.9.2 Proofs of Inclusion . . . 18

2 Foundational Quantum Algorithms **23**
2.1 Grover's Algorithm . . . 23
2.1.1 The Unstructured Search Problem . . . 23
2.1.2 Algorithm Structure . . . 23
2.1.3 Geometric Proof of Correctness . . . 24
2.1.4 Synthesis and Implementation . . . 27
2.2 Advanced Variants of Grover's Algorithm . . . 28
2.2.1 Exact Quantum Search . . . 28
2.2.2 Fixed-Point Quantum Search . . . 31

Chapter 1

Theoretical Foundations of Quantum Computation

1.1 Introduction

Information processing is one of the most common problems of our times.

For decades, the engineers has relied on the predictable, deterministic laws of classical physics to manipulate discrete zeros and ones inside the computers. Current computers are still improving their computational power and it is famously characterized by Moore's Law, involved shrinking transistors year after year to pack more computational power onto a single chip. However, as these electronic components reach the atomic scale, the classical physics models begin to break down, as all in practice have their limit in size. At the sub-nanometer level, quantum mechanical phenomena like electron tunneling cease to be minor engineering nuisances and instead become dominant forces that actively disrupt classical logic operations.

In the early 1980s, physicists like Paul Benioff [Ben80] and Richard Feynman [Fey82] realized that rather than fighting these quantum effects, we could try to harness them. Feynman noted a profound asymmetry in computation: simulating quantum mechanical systems on classical hardware inherently requires an exponential amount of memory and processing time. His intuition was like: If nature operates quantum mechanically, a computer natively built from quantum components could simulate nature or it's effects more efficiently. This physical intuition lead to creating new computing model, Universal Quantum Turing Machine [Deu85], formalized by David Deutsch in 1985. Deutsch mathematically proved that a quantum machine could perform any computation a classical one could and probably use some non-classical algorithmic pathways.

The building block of this quantum computing is qubit, which is the object corresponding to the bit in classic computing. Unlike a bit, which behaves like a standard switch being only in a 0 or 1 state, a qubit can exist in a *superposition* of states (it's unknown state). A helpful intuition is to think of superposition not as "being in two states at once", but rather as a wave-like probability distribution where the exact state remains fluid until it is measured. Furthermore, multiple qubits can experience *entanglement*, a situation where

their quantum states become related and the information of the overall system is stored primarily in the mathematical relationships between the qubits, rather than within the individual qubits themselves. These two properties allow quantum algorithms to map, explore vast and complex state spaces in ways normal computers could have problems.

When Peter Shor published his quantum algorithm for integer factorization [Sho94] in 1994, the quantum algorithms interest suddenly rised up and the goal of creating a quantum computer and utilize it's full possibilities became even more tempting. Shor's algorithm proves that a quantum computer could efficiently solve specific algebraic problems considered time consuming for classical approaches. This discovery was really great, because it meant quantum advantage was not limited to simulating physics and it could have real-world application capable of breaking the RSA cryptographic infrastructure that e.g. secures modern digital communication.

To fully understand how algorithms like Shor's and Grover's works, one must first understand basics of quantum computing. This chapter contains the theoretical background necessary to follow selected examples and algorithms. We begin by covering the linear algebra that governs quantum mechanics, define the qubit and entangled systems and finally define the quantum circuit model and the formal complexity classes that define quantum advantage.

1.2 Mathematical Prerequisites

To get deeper into quantum algorithms, one must first understand in the mathematical language of quantum mechanics, which is linear algebra.

Unlike classical computing systems, whose states are described by discrete deterministic variables, quantum systems are probabilistic. Which causes, that their foundational behavior is formalized using complex vector spaces, inner products, and linear operator theory [NC10].

1.2.1 Hilbert Spaces and Dirac Notation

The state of any isolated quantum system is completely described by a unit vector residing in a complex *Hilbert space*, denoted by $\mathcal{H}$. Hilbert space can be thought of as a generalization of a standard Euclidean space. The familiar geometric notions of length and angle still apply, but the space is extended to accommodate complex numbers. For the finite dimension systems relevant to quantum computing one, such as a register of n interacting qubits, the Hilbert space is isomorphic to $\mathbb{C}^{2^n}$. This means that it is 2^n dimensional complex vector space and the exponential growth factor hints at the parallel processing power of quantum computing.

To manipulate these state vectors, we use Dirac's *bra-ket* notation [Dir39]. It's provides a algebraic shortcut that maps perfectly to the underlying linear algebra operations:

- **Ket** ($|\psi\rangle$): A standard column vector representing a distinct quantum state.

- **Bra** ($\langle\psi|$): The Hermitian conjugate (conjugate transpose) of the ket, representing a row vector residing in the dual space $\mathcal{H}^*$.

For a vector $\psi \in \mathbb{C}^d$, we write:

$$
\begin{aligned}
|\psi\rangle &= \begin{pmatrix} z_1 \\ \vdots \\ z_d \end{pmatrix}, \\
\langle\psi| = |\psi\rangle^\dagger &= \begin{pmatrix} z_1^* & \dots & z_d^* \end{pmatrix}
\end{aligned}
\tag{1.1}
$$

where, z^* is the complex conjugate of z.

1.2.2 Inner and Outer Products

The **inner product** of two state vectors $|\phi\rangle$ and $|\psi\rangle$ is a scalar $c \in \mathbb{C}$, denoted by the combined bracket $\langle\phi|\psi\rangle$. This operation measures the "overlap" or similarity between two quantum states and it satisfies the following properties:

1. Conjugate symmetry: $\langle\phi|\psi\rangle = \langle\psi|\phi\rangle^*$.
2. Linearity in the second argument: $\langle\phi| (a|\psi_1\rangle + b|\psi_2\rangle) = a\langle\phi|\psi_1\rangle + b\langle\phi|\psi_2\rangle$.
3. Positive definiteness: $\langle\psi|\psi\rangle \geq 0$, with equality if and only if $|\psi\rangle = 0$.

Two vectors are considered *orthogonal* when their inner product is equal to 0 ($\langle\phi|\psi\rangle = 0$). This condition signifies that they represent mutually exclusive quantum states. The norm corresponds to the geometric length of a state vector and it is defined as $||\psi|| = \sqrt{\langle\psi|\psi\rangle}$. For any vector to represent a valid quantum state, it must be properly normalized to unity, which mean that it's norm have to be equal to one ($||\psi|| = 1$). Normalizing the state vector ensures that all possible measurement probabilities will always sum to exactly 100%.

The **outer product** of $|\psi\rangle$ and $|\phi\rangle$ forms a linear operator (a matrix) denoted as $A = |\psi\rangle\langle\phi|$. Applying this operator to a new vector $|\xi\rangle$ yields a vector proportional to $|\psi\rangle$:

$$
(|\psi\rangle\langle\phi|)|\xi\rangle = |\psi\rangle(\langle\phi|\xi\rangle) = c|\psi\rangle \tag{1.2}
$$

where $c = \langle\phi|\xi\rangle$ is scalar multiplier.

Outer products are utilized in quantum algorithms to construct projection operators, which map directly to quantum measurements.

1.2.3 Linear Operators

Quantum states do not exist statically, as they constantly evolve. This evolution is described by linear operators acting upon the Hilbert space $\mathcal{H}$.

An operator A is considered linear if it preserves the principle of superposition, meaning $A(c_1 |\psi_1\rangle + c_2 |\psi_2\rangle) = c_1 A |\psi_1\rangle + c_2 A |\psi_2\rangle$.

Hermitian Operators and Observables

An operator A is *Hermitian* (or self-adjoint) if it is equal to its conjugate transpose, $A = A^\dagger$. The Spectral Theorem states that for any Hermitian operator M on a finite dimensional space, there inherently exists an orthonormal basis of eigenvectors $\{|m_i\rangle\}$ paired with purely real eigenvalues m_i such that:

$$M = \sum_i m_i |m_i\rangle \langle m_i| \tag{1.3}$$

This decomposition has a physical consequences. The result of measuring the observable M will always be one of those real values m_i.

Unitary Operators and Evolution

An operator U is *unitary* if its inverse is exactly equal to its adjoint:

$$U^\dagger U = UU^\dagger = I \tag{1.4}$$

Unitary operators preserve the inner product: $\langle\psi| U^\dagger U |\phi\rangle = \langle\psi|\phi\rangle$, which means they preserve the norm (length) of any state vector they act upon. This preservation is essential for quantum computing, because the norm corresponds directly to total probability. Application of an unitary operator acts as a guarantee that the sum of probabilities for all possible outcomes remains exactly 1 throughout the entire duration of the computation. Therefore, all logic gates applied to a quantum register must be formulated as unitary matrices [NC10].

1.3 The Qubit

The fundamental unit of quantum information is the *qubit* (quantum bit) [NC10]. While a bit is a logical abstraction realized physically by a system with two distinct stable states (such as a high or low electrical voltage), a qubit is a two-level quantum mechanical system. Physical realizations of a qubit may include the spin configuration of a single electron or the polarization state of a photon.

1.3.1 Computational Basis

The state of a single qubit is represented by a unit vector from two dimensional complex Hilbert space, $\mathbb{C}^2$. We define an orthonormal basis $\{|0\rangle, |1\rangle\}$, which is our *computational basis*. These vectors correspond directly to the deterministic logical states 0 and 1:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \tag{1.5}$$

1.3.2 Superposition

Unlike the bit, which must be in one state or the other, a qubit can exist in a linear combination of its basis states. This property is known as *superposition*. It allows quantum algorithms to process many informations simultaneously. A generalized qubit state $|\psi\rangle$ is described by the equation:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle \tag{1.6}$$

where α, β are complex numbers known as *probability amplitudes*.

According to the Born rule, if we measure a qubit, the superposition will collapse into one of the definite basis states ($|0\rangle$ or $|1\rangle$). The probability of the $|0\rangle$ is $P(0) = |\alpha|^2$, and the probability of $|1\rangle$ is $P(1) = |\beta|^2$, which in total gives us that it must sum to exactly 100%. This forces the state to be bound by the normalization condition:

$$|\alpha|^2 + |\beta|^2 = 1 \tag{1.7}$$

1.3.3 The Bloch Sphere

As α and β are complex numbers, a qubit state in thoery have four real parameters, but now using the normalization condition we can write a single qubit using only three real parameters:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) e^{i\gamma_\alpha}\,|0\rangle + \sin\left(\frac{\theta}{2}\right) e^{i\gamma_\beta}\,|1\rangle \tag{1.8}$$

We can cut it just to two parameters using the global phase. We will factor out the phase of the first term, $e^{i\gamma_\alpha}$ from the expression:

$$|\psi\rangle = e^{i\gamma_\alpha}\left(\cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i(\gamma_\beta-\gamma_\alpha)}\sin\left(\frac{\theta}{2}\right)|1\rangle\right) \tag{1.9}$$

The term $e^{i\gamma_\alpha}$ before the parentheses is known as the global phase factor of the quantum state.

As we calculate the probability of measurement, the global phase is multiplied by its conjugate:

$$|e^{i\gamma\alpha}|^2 = e^{i\gamma\alpha}\cdot e^{-i\gamma\alpha} = e^0 = 1 \tag{1.10}$$

Because of this, we can define ϕ, as $\phi = \gamma_\beta - \gamma_\alpha$ and simplify the parameterization of qubit state using only two real angles, θ and ϕ:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle \tag{1.11}$$

The polar angle $0 \leq \theta \leq \pi$ is responsible for probability distribution of $|0\rangle$ and $|1\rangle$ outcomes. The azimuthal angle $0 \leq \phi < 2\pi$ defines the *relative phase* between the two states and will be really useful factor in quantum algorithms (e.g. Grover's algorithm). Now we can map every qubit state to a point on the surface of a unit sphere, known also as the **Bloch Sphere** [NC10].

- The North Pole ($\theta = 0$) represents the state $|0\rangle$.
- The South Pole ($\theta = \pi$) represents the state $|1\rangle$.
- Points lying perfectly on the equator (e.g., $\theta = \pi/2, \phi = 0$) represent equal superpositions. Where, the qubit has a 50% chance of collapsing into either basis state upon measurement.

Bloch sphere provides an visual intuition for single qubit operations, as they are just rotations on the sphere, however it fails to generalize to systems with many qubits due to non-local nature of quantum entanglement.

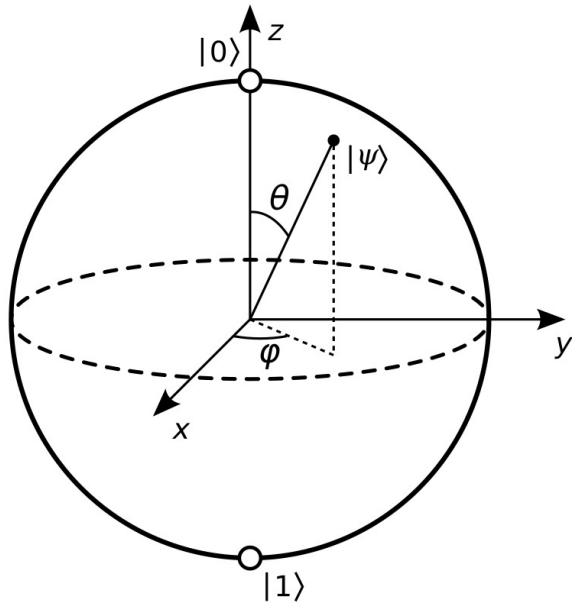

Figure 1.1: Example Bloch sphere. Source: Wikipedia.

1.4 Composite Systems and Tensor Products

While a single qubit provides the baisc building block to use in quantum computing, full capabilities are achieved when multiple qubits are used and allowed to interact. To describe the state of many quibits, we use the *tensor product* (also knownas the Kronecker product), denoted by the symbol $\otimes$ [NC10]. If we have a system A in a Hilbert space $\mathcal{H}_A$ of dimension N, and a second system B in Hilbert space $\mathcal{H}_B$ of dimension M, then their composed system AB is in the tensor product space $\mathcal{H}_A \otimes \mathcal{H}_B$. The total dimension of this combined space is $N \times M$, rather than $N + M$ typically seen composing to spaces. For example having two separate qubits, where both are in $\mathbb{C}^2$, the composite two qubit state is in a four dimensional complex vector space, $\mathbb{C}^2 \otimes \mathbb{C}^2 \cong \mathbb{C}^4$.

The computational basis for this two qubit system is created by taking every possible pairs of the basis vectors:

$$\{|0\rangle \otimes |0\rangle, |0\rangle \otimes |1\rangle, |1\rangle \otimes |0\rangle, |1\rangle \otimes |1\rangle\} \tag{1.12}$$

We can simplify the notation of many qubit systems, as we usually skip the tensor product

between the states and write them simply as $|00\rangle, |01\rangle, |10\rangle$, and $|11\rangle$. The most general state of a two qubits, $|\psi\rangle$, is a superposition of all four of basis states:

$$|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle \tag{1.13}$$

Same as with the single qubit, the system with many qubits must hold the sum of the

squared probability amplitudes equal to 100%: $\sum_{i,j\in\{0,1\}} |\alpha_{ij}|^2 = 1$.

This clearly shows the interesting property of many qubit system. Adding just a single qubit to a register doubles the total number of amplitudes required to fully track the system's state. For a register of n qubits, the state space consist of 2^n complex dimensions. This exponential growth shows the possible parallelism of quantum algorithms, representing a possible source of their "theoretical" computational advantage over classical algorithms [AB09].

1.5 Quantum Entanglement

When many qubits are combined, their joint state space is constructe by the tensor product of their spaces. This structure gives rise to states that have no classical equivalent. A state with many qubits is defined as *separable* (or a product state) if it can be factored into the tensor product of independent qubit states.

For instance, the uniform superposition state $|\psi\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$ is separable. This is because it can be factored into $\left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\right) \otimes \left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\right)$. That means changing or measuring one qubit does not force any outcome on the other qubits. However, lot of states from composite Hilbert space cannot be factored in this way. States that cannot be factored in that way are called *entangled*. Entanglement is a form of non local correlation that is used very often in quantum algorithms. In a entangled system, the quantum state of the is whole system may be defined, yet the states of the subsystems which it consists of may still be undefined. This non-classical behaviour was famously, described by Albert Einstein, Boris Podolsky, and Nathan Rosen (EPR) as "spooky action at a distance" [EPR35].

1.5.1 The Bell States

The most iconic examples of entangled two qubits are the four Bell states, also often mentioned in the literature as EPR pairs [NC10]. They form a orthonormal basis for the two qubits state space and are defined as follows:

$$\left|\Phi^+\right\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \tag{1.14}$$

$$\left|\Phi^-\right\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) \tag{1.15}$$

$$\left|\Psi^+\right\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) \tag{1.16}$$

$$\left|\Psi^-\right\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \tag{1.17}$$

It is easy to observe that summing the first two equations gives us $\frac{2}{sqrt2}(|00\rangle)$. In similar ways we can get any of the possible states.

To give some intuitions, let's consider the specific entangled state $|\Phi^+\rangle$. Imagine also two experimental parties, named Alice and Bob, each of them having one qubit of this pair. If Alice will measure her qubit, she will faces a 50% probability of observing a $|0\rangle$ and a 50% probability of observing a $|1\rangle$. However, after that is the twist. The moment she observes a $|0\rangle$ (the case will be similarly with $|1\rangle$), the global wave function of the system collapses instantly from the superposition state into the definite state $|00\rangle$. If Bob now measures his qubit, he will for sure observe a $|0\rangle$ with 100% certainty. This correlation holds true regardless of whether Bob and Alice are in the same place or far way. The main factor is that in that case, this qubits states are strongly connected to each other via system they are part of (which in this case was $|\Phi\rangle^+$).

This correlation violates the limits established by Bell's inequalities, which is quite important as it proves that quantum mechanics cannot be explained or modeled by "local hidden variable" theories [Bel64]. In other words, the measurement outcomes are not secretly assigned before they are separated.

Due to the fact that the entangled system cannot be split into independent states, the Bloch sphere for single qubit, fails while trying to visualize the entanglement. It is the classical physical paradox here. While we can possess full knowledge about the joint system as a whole, we possess no knowledge about its individual parts before the measurement.

1.6 Quantum Logic Gates and Circuits

Just as classical algorithms on its basis are constructed with logic gates like AND, OR, NOT and so on, quantum algorithms are built using *quantum circuit model*. In quantum computations we apply a sequence of operations, known as quantum logic gates, to a set of initialized qubits. The computation then finishes with a measurement to extract the final state and gives us the desired result [Deu89]. A main rule of quantum systems is that their evolution in time is deterministic and reversible, because of that, each quantum gate must be an invertible operation. The only exception to that rule is the measurement operation, which irreversibly collapses the state and provides our result. Additionally, because these gates manipulate probability amplitudes, they must preserve the fundamental rule that all probabilities sum exactly to 100%. To satisfy these twin requirements of reversibility and probability conservation, all quantum gates are mathematically represented as *unitary matrices* [NC10].

1.6.1 Single-Qubit Gates

Gates for a single qubit operate on the state space $\mathbb{C}^2$, and can be represented as 2×2 unitary matrices. The most known and useful of gates for a single quibit are three Pauli matrices. They correspond to 180° degree (π radian) rotations around each of axes (x, y,

and z) of the Bloch sphere:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \tag{1.18}$$

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \tag{1.19}$$

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \tag{1.20}$$

The X gate is the equivalent of the NOT gate in quantum computing. It flips the sate of the quibit which maps $|0\rangle \to |1\rangle$ and $|1\rangle \to |0\rangle$. The Z gate on the other hand is like a phase flip. It leaves the $|0\rangle$ state unchanged, but flips the sign of $|1\rangle$, mapping it to $-|1\rangle$. Gate Y works on a quibit as gate X fliping the state of the quibit but also applyes the complex phase shift to it mapping $|0\rangle \to i|1\rangle$ and $|1\rangle \to -i|0\rangle$. Probably the most common operation amongs the operations on a single qubit is the Hadamard gate, denoted H. It is a basic gate for generating quantum superposition from the input state and typically occurs right on the start of the circuit. It creates a equal probability distribution between $|0\rangle$ and $|1\rangle$ state and can be denoted as:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \tag{1.21}$$

Applying H to $|0\rangle$ yields the positive superposition state $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and applying

it to $|1\rangle$ yields the negative superposition state $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ [Mer07]. Other useful operations are the Phase gate (S) and the $\pi/8$ gate (T). Instead of flipping the state (like the X gate), these gates rotate it around the Z-axis (similarly to the Z), changing the relative phases without changing the measurement probabilities. This gate can be denoted as:

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} \tag{1.22}$$

1.6.2 Two Qubit Gates: CNOT, SWAP, and Controlled-Phase

While gates that operates on single qubit can modyfy it state spaces (probabilities and phase), we must now be able to apply gates operating on multiple qubits to be able to perform conditional logic or generate entanglement. The basic operation on two qubits is the Controlled-NOT (CNOT) gate.

We can say, that the CNOT gate works with a control qubit and a target qubit. If the control qubit is in the $|1\rangle$ state, the gate applies the X gate to the target qubit (it

flips a bit of target qubit). If the control qubit is in the $|0\rangle$ state, the target qubit is left completely untouched.

In the standard computational basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$, where the first qubit acts as the control, the matrix representation of CNOT looks like:

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{1.23}$$

The CNOT gate is really useful and will serves as the basic mechanism for entangling previously independent qubits. For example, if we start with the state $|00\rangle$, then we apply a Hadamard gate to the first qubit to create a superposition state with equal probabilities, and then apply a CNOT gate, we generate the entangled Bell state $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ [NC10].

Next really practical and usefull operation on two quibits is the SWAP gate. It exchanges the quantum states of two distinct qubits.

This gate is usefull for routing information across physical hardware topologies, as not all qubits are directly connected to each other. Typically only the nearest ones are only connected so we have to change theirs positions. It also will act as the final bit reversal step of the Quantum Fourier Transform (QFT) used in Shor's algorithm:

$$\text{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{1.24}$$

While the CNOT conditionally a flip a bit or leaves it unchanged, the Controlled-Phase gate, denoted as $CP(\theta)$ or $CR_z(\theta)$ applies a conditional phase shift. This gate adds a complex phase of $e^{i\theta}$ to the system if, and only if, both the control and target qubits are simultaneously in the $|1\rangle$ state:

$$CP(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\theta} \end{pmatrix} \tag{1.25}$$

This gate enables the precise extraction of periodic phases, which will necessary for tasks like integer factorization (Shor's algorithm).

1.6.3 Advanced and Multi-Controlled Gates

To create more complex logical structures, like the Search Oracles in Grover's algorithm or the modular arithmetic in Shor's algorithm. We need gates that condition their behavior on several qubits at once.

The Toffoli gate, also referred to as the Controlled-Controlled-NOT (CCNOT), is the most famous example of this. It works quite simple. It observes two control qubits and applies an X gate to a single target qubit if, and only if, both controls are in the $|1\rangle$ state. Because the Toffoli gate is proven to be universal for classical reversible computation, it is the key gate for building quantum oracles that will evaluate classical boolean functions [NC10]. The CCNOT gate can be denoted as:

$$\text{CCNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \tag{1.26}$$

For more advanced quantum algorithms, the Toffoli gate is often generalized into the Multi-Controlled X gate (MCX). The MCX gate flips a target qubit conditioned on the state of an arbitrary number of n control qubits.

Similarly, the diffusion step in Grover's algorithm relies heavily on a Multi-Controlled Z gate (MCZ), which is instead of flipping the bit (applying the X gate), the MCZ gate flips the phase (the sign, by applying the Z gate), but only when all n qubits in the same time are in the $|1\rangle$ state.

In practice we may have not ready native MCZ gate availbale to us, but we can easily synthesize one by taking an MCX gate and wrapping its target qubit in Hadamard gates. This trick smoothly shifts the bit flip operation into a phase flip operation: $MCZ = (I \otimes \cdots \otimes H)MCX(I \otimes \cdots \otimes H)$ [Bar+95].

Many algorithms may also rely on the generalized Controlled-U operation ($c - U$). Imagine an arbitrary quantum operation U designed to act on a large target set of n qubits. The $c - U$ gate check if a single control qubit is equal to $|1\rangle$, and if so, it triggers the U operation on the target qubits.

1.7 Universal Quantum Computation

In classical computing, the idea of universal computation is quite simple. A small, finite set of logic gates, or even just one gate like NAND, is enough to build and construct any possible boolean function.

Meanwhile in the quantum computing, we meet a big problem on our way. The quantum state space is continuous. Valid quantum operations are rotations in a Hilbert space, due to that trying to hit a specific target state become quite problematic. To execute exact, arbitrary unitary transformations, we would in theory need an infinite set of logic gates which could be impossible for us.

To fix this gap between continuous unitary operations and the discrete hardware, quantum algorithms use the Solovay-Kitaev theorem [DN05].

This theorem says that we do not need an infinite amount of gates to move through the state space and perform our operation. Instead of that, we can represent any arbitrary quantum operation as an approximated one with some desired accuracy, ϵ, using only finite set of basic quantum gates. What is also important, it describes that the number of basic gates needed to get this approximation scales only polylogarithmically with $1/\epsilon$. This small cost ensures, that approximating the operations will not destroy our time complexity of algorithms.

Due to that problem, modern fault tolerant quantum computers use the Clifford+T universal gate set. This standard library consists of the Hadamard (H), the $\pi/8$ phase gate (T), and the two-qubit CNOT gate. We can also observe an interesting fact on this set. Circuits built entirely of Clifford gates (like H, CNOT and $\pi/8$) are easy to simulate on a standard computers. This limit is formalized by the Gottesman-Knill theorem [NC10]. However, after adding T gate to the set, it breaks this classical simulation. This also allows us to move freely on the continuous quantum space, giving this set a full quantum universality.

1.8 Theoretical Machine and Memory Models

Before we can start to analyze how fast or efficient are quantum algorithms, we first need to define the theoretical models (computing model) it could runs on.

The quantum circuit computing model is great for visualizing and designing algorithms, it is more intuitive and easier to write in a readable for, but formal computer science uses the abstract Quantum Turing Machine (QTM). Proving that these two computing models are equivalent in terms of the computing capabilities is what allows us to use quantum circuits to solve problems [BV97].

1.8.1 The Quantum Turing Machine (QTM)

The Quantum Turing Machine (QTM) is the basic theoretical model for quantum computation. It was initially designed by Benioff and Feynman in the early 1980s and formalized by Deutsch in 1985 [Deu85], and later put into formal computational complexity classes by Bernstein and Vazirani [BV97].

Similarly to classic Turing machine (TM), a QTM is defined by a 7-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject}) \tag{1.27}$$

- Q is the finite set of internal machine states.
- Σ is the initial input alphabet.
- Γ is the tape alphabet, which includes Σ and a blank symbol.
- $q_0, q_{accept}, q_{reject} \in Q$ are the start, accept, and reject states.

The difference between a TM and QTM in the transition function δ. Instead of moving us to a single new state, a QTM transitions into a quantum superposition of states. To be more precise δ assigns a complex amplitude to every possible move:

$$\delta : Q \times \Gamma \times Q \times \Gamma \times \{L, R\} \to \mathbb{C} \tag{1.28}$$

For a QTM to represent a real physical system, its path of computation in time must be completely reversible. This means the transition operator U must be unitary ($U^\dagger U = I$). Unlike in classic machines, a QTM's amplitudes must follow strict rules to ensure that the global probability always sums exactly to 100% [BV97].

If we pause or stop a running QTM at any moment, it is not just in one specific tape configuration. It exists in a big, entangled superposition of the tape contents, head position, and its internal state. When we at the end measures the state of the machine, the probability of seeing any specific setup is given by the Born rule.

Also by proving that a single "Universal" QTM (similar to normal TM) can simulate any other QTM, researchers created the groundwork for defining what quantum computers can solve in practice.

1.8.2 Equivalence of QTMs and Quantum Circuits

While the QTM is an ideal framework for formal proofs, it is hard to use for algorithm design. Due to that, almost all quantum algorithms are written using the much more intuitive quantum circuit model. The reason we can switch between these two models without loosing any formality it is because of a theorem by Andrew Yao in 1993 [Yao93].

Theorem (Yao's Equivalence): Any computation that successfully halts in time $T(n)$ on a Quantum Turing Machine can be simulated by a quantum circuit using at most $\mathcal{O}(T(n)^2)$ gates. Also in the other direction, any quantum circuit with $S(n)$ gates can be simulated by a QTM in a time bounded by a polynomial of $S(n)$.

Simulating a QTM with a Circuit (Proof outline): Let's take a QTM that runs for $T(n)$ steps. In that time, its tape head can only move at most $T(n)$ cells away from the start. Because the space is bounded from above, we can constructe a quantum circuit where each qubit represents a specific cell on that tape. Since the QTM reads exactly one

cell at a time, a single step of the machine translates to a small layer of quantum logic gates.

To simulate the run from the beggining to the end, we will just connect $T(n)$ of these gate layers together. To handle the fact that the tape head might be in a superposition of different locations, we use controlled-SWAP gates to route information. This ensures the final circuit size only grows to about $\mathcal{O}(T(n)^2)$.

Simulating a Circuit with a QTM (Proof outline):

Going in the other way, a Universal QTM can act as a circuit interpreter.

First, the machine computes the blueprint of the required circuit. The QTM then uses its tape to hold the logical states of the circuit's qubits.

It reads the generated blueprint one gate at a time. To apply a gate, the QTM moves its head to the correct "qubit" cells and applies the unitary matrix. Moving the head back and forth to connect non-adjacent qubits takes some time, but this time only scales polynomially. Because of this, the simulation is efficient. ■

1.8.3 Quantum Random Access Memory (QRAM)

A thing worth to mention is Quantum Random Access Memory (QRAM). While the QTM and circuit models cover standard operations, some new algorithms need something more.

Algorithms like HHL for solving big linear equations [HHL09] and quantum machine learning need to process big amounts of classical data.

To load this data without destroying their quantum speedups, they rely on a specialized theoretical hardware called QRAM [GLM08].

Classic RAM lets a computer get one specific piece (sequential pack) of data from a single memory address (and some of it's sourroundings).

QRAM allows a quantum computer to query a superposition of addresses in the same time. Given an address register A and a targeted data register D, an ideal QRAM performs the following elegant operation:

$$\sum_{i=0}^{N-1} \alpha_i \left|i\right\rangle_A \left|0\right\rangle_D \xrightarrow{\text{QRAM}} \sum_{i=0}^{N-1} \alpha_i \left|i\right\rangle_A \left|x_i\right\rangle_D \tag{1.29}$$

where, x_i encodes the classical data stored in the i-th memory slot.

[Not fully feeling that, include it all here or skip it? (I mean the below)]

The engineering challenge behind building a QRAM is really big. A simple design would wire the address register directly to all N memory cells at once. In the quantum world, this approach will be a disaster. Forcing the address qubits to interact with millions of logic gates in the same time opens the door to noise from the environment, causing the superposition to collapse instantly, which is known as decoherence.

To solve this problem, Giovannetti et al. proposed a better way: the *bucket-brigade architecture* [GLM08]. Instead of a massive connection, the memory is organized into a binary routing tree.

When a quantum address goes into the root of the tree, it acts like a train that flips the

switches along its specific path. It shifts the required nodes from a disconnected "wait" state into an active state. The most important thing in this design is isolation. Only the specific nodes on the active path are "woken up" by the system. The rest of the tree remains asleep and completely isolated from environmental noise.

Because of this, accessing an array of N items only activates $\mathcal{O}(\log N)$ quantum switches per query, rather than N. This architecture reduces the decoherence, making big data queries possible for future hardware.

1.9 Computational Complexity and Quantum Advantage

[Przepisz to] To really understand what makes a quantum computer computationally "powerful," we need to define the mathematical boundaries of what it can actually do. Instead of relying on vague claims, we place quantum algorithms into the well-established framework of classical computational complexity theory [AB09]. This gives us an objective, rigorous way to compare how quantum algorithms scale against the absolute best classical methods as the size of a problem grows.

1.9.1 Definitions of Complexity Classes

Before we can map out the territory of quantum advantage, we need to define our key complexity classes. Think of these classes as buckets that group computational problems based on how quickly their time and memory requirements blow up as the size of the input, N, gets larger.

- **P (Polynomial Time):** The bread and butter of classical computing. This is the class of problems that a standard computer can solve efficiently, meaning the runtime scales gracefully (like N^2 or N^3).

- **ZPP (Zero-error Probabilistic Polynomial time):** Imagine a classical computer with a random coin. It uses randomness to speed things up, and when it finally gives an answer, it is always 100% correct. However, we only guarantee that it finishes quickly *on average*. Formally, $\text{ZPP} = \text{RP} \cap \text{co-RP}$.

- **RP (Randomized Polynomial time):** A classical algorithm that runs quickly and uses randomness, but is slightly pessimistic. If the true answer is "no", it always correctly says "no". If the true answer is "yes", it might accidentally say "no", but it has at least a 50% chance of getting the "yes" right.

- **BPP (Bounded-error Probabilistic Polynomial time):** This is the gold standard for practical classical algorithms. The computer runs quickly but might make a mistake on both "yes" and "no" answers. However, the probability of being right is always bounded strictly away from a coin flip (typically defined as $\geq 2/3$). Because it's reliably better than 50/50, we can just run the program a few times and take a majority vote to confidently find the truth [AB09].

- **NP (Nondeterministic Polynomial time):** These are problems that might be incredibly hard to solve from scratch, but if someone hands you the correct answer (a "witness"), you can verify it very quickly. Think of a Sudoku puzzle: hard to fill out, but trivial to check if it's right.

- **BQP (Bounded-error Quantum Polynomial time):** The quantum sibling to BPP. This is the exact class of problems that a quantum computer can solve efficiently. It allows for a small margin of error (at most 1/3), but just like BPP, we can easily shrink that error to near zero by running the quantum circuit a few extra times and taking a vote [BV97; NC10].

- **PP (Probabilistic Polynomial time):** A bizarre and incredibly powerful theoretical class. A classical computer guesses the answer, and it only needs to be right strictly more than 50% of the time—even if it's 50.0000001%. Because the margin is so razor-thin, taking a majority vote would physically require an exponential number of runs. Despite being physically impractical, it is a mathematical behemoth that easily swallows up both NP and BQP [ADH97].

- **PSPACE (Polynomial Space):** Problems that can be solved using a reasonable (polynomial) amount of memory. The catch? There is no time limit, meaning the computer might take until the heat death of the universe to finish its calculation.

1.9.2 Proofs of Inclusion

Now that we have our classes, we can conceptually stack them like Russian nesting dolls: P $\subseteq$ ZPP $\subseteq$ RP $\subseteq$ BPP $\subseteq$ BQP $\subseteq$ PP $\subseteq$ PSPACE. While almost every computer scientist firmly believes that BQP is strictly mathematically larger than BPP—implying quantum computers can efficiently solve problems that classical computers inherently cannot—proving this strict separation remains one of the greatest open problems in theoretical computer science. We can, however, definitively prove the inclusion boundaries. Let's walk through the intuition for why these theoretical dolls fit inside each other.

Theorem 1: P $\subseteq$ BPP

Proof. This one is almost trivially true. Suppose we have a problem that can be solved perfectly by a standard deterministic computer (Class P). We can simply hand this exact same problem to a randomized computer (Class BPP) and instruct it to never roll its dice. Because the program never uses randomness, it behaves exactly like the deterministic machine, succeeding with 100% certainty. Since 100% is well above the 66% success threshold required by BPP, the problem comfortably fits inside it. □

Theorem 2: BPP $\subseteq$ BQP

Proof. Can a quantum computer simulate a randomized classical one? Yes, and the process requires exactly two conceptual steps.

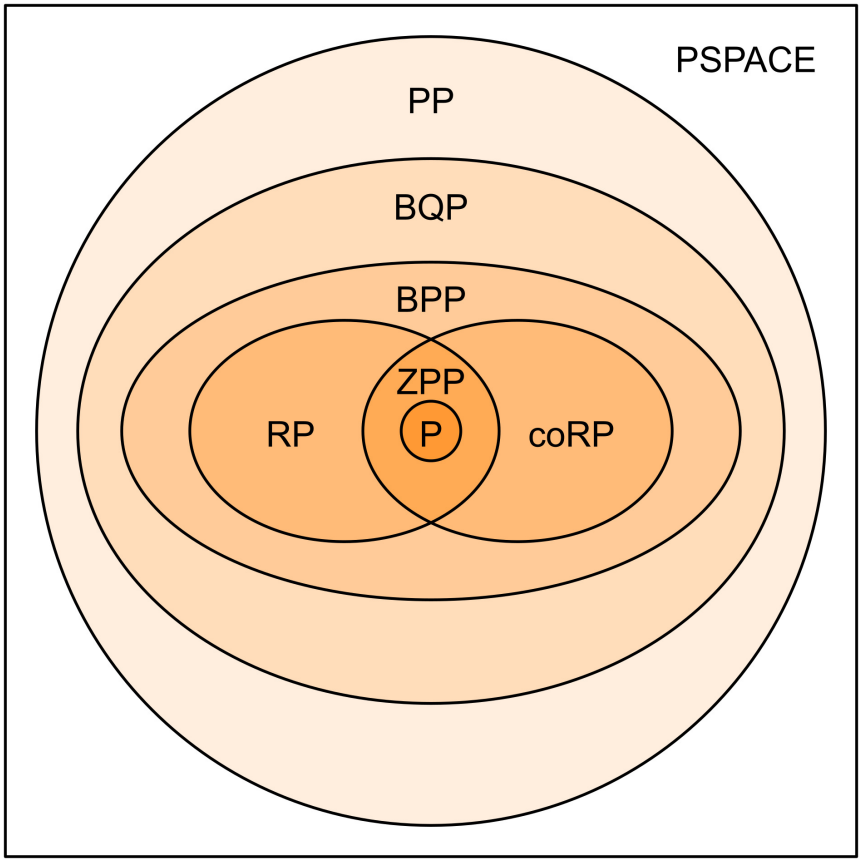

Figure 1.2: Euler diagram illustrating the theoretical relationships and hierarchy of classical and quantum computational complexity classes. Source: Wikipedia

First, we must deal with the fact that quantum mechanics is fundamentally reversible. Standard classical computers throw away information (for example, an AND gate irreversibly crushes two bits into one). However, Bennett's theorem mathematically proves that any classical program can be rewritten into a perfectly reversible one using energy-conserving logic like the Toffoli gate, incurring only a tiny structural overhead [Ben73].

Second, the classical BPP machine needs random coin flips to operate. A quantum computer is arguably nature's perfect random number generator. By taking a fresh qubit in the $|0\rangle$ state and hitting it with a Hadamard gate, we create a perfect 50/50 superposition:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \tag{1.30}$$

Measuring this qubit yields a perfectly unbiased, fundamentally unpredictable random bit.

By feeding a steady stream of these quantum coin flips into our reversible classical circuit, the quantum computer flawlessly mimics the classical BPP machine. It preserves the exact same probability of success, definitively proving that a quantum computer can

do everything a randomized classical computer can do. □

Theorem 3: BQP $\subseteq$ PSPACE

Proof. This theorem asks a fascinating question: could a classical computer simulate a quantum one if we gave it unlimited time, but strictly limited memory? The answer is yes, using an intuitive method inspired by Richard Feynman's sum-over-histories [BV97].

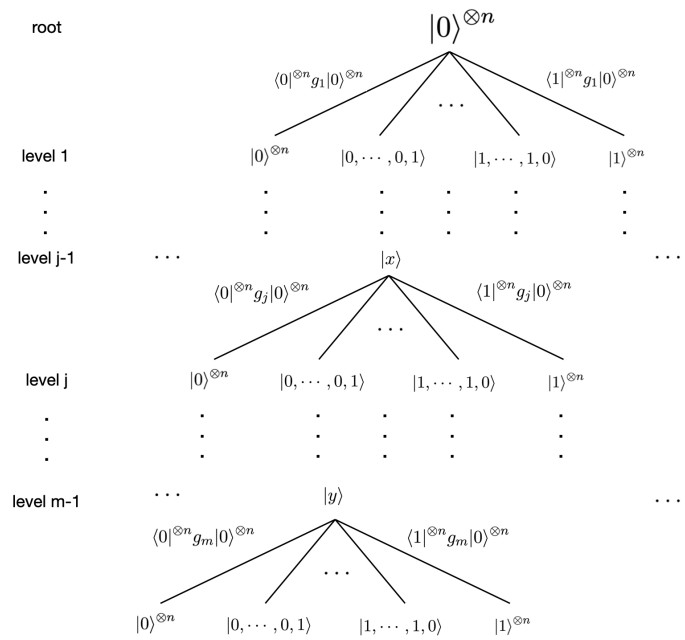

Figure 1.3: A computational tree representing the Feynman sum-over-histories approach for evaluating the probability amplitudes of quantum circuits. Source: Wikipedia

Imagine the execution of a quantum circuit as a massive branching tree of possibilities. If a circuit is made of m sequential gates $U_1, U_2, \ldots, U_m$, the transition amplitude from a starting state $|x\rangle$ to a final state $|y\rangle$ is mathematically defined as:

$$\langle y| U_m U_{m-1} \ldots U_1 |x\rangle \tag{1.31}$$

By breaking this down, we can view it as a sum over every possible intermediate state, z, at every step of the computation:

$$\sum_{z_1, \ldots, z_{m-1}} \langle y| U_m |z_{m-1}\rangle \langle z_{m-1}| U_{m-1} |z_{m-2}\rangle \ldots \langle z_1| U_1 |x\rangle \tag{1.32}$$

At every single gate operation, the universe of the simulation splits into different paths, each carrying a tiny piece of this complex probability amplitude. Because a quantum register with n qubits evaluating m gates creates 2^{nm} possible branches, a classical computer would take practically forever to calculate this massive sum.

However, it does not need to store that entire branching universe in its RAM at once. Instead, it can use a simple depth-first search. The classical computer traces one single, isolated path all the way from the root to the leaf, calculates its specific amplitude, adds it to a running numerical tally, and then completely overwrites that path in memory to calculate the next one.

To pull this off, the classical computer only needs to remember two things: the chronological sequence of the current path, and the running probability total. Because the circuit size is polynomial, both of these metrics require very few bits to track—scaling only polynomially with the size of the input. Since the active memory ceiling stays low even as the execution time stretches toward infinity, the entire simulation comfortably operates within PSPACE. □

Theorem 4: BQP $\subseteq$ PP

Proof. As we just saw, calculating a quantum probability requires adding up a staggering number of paths. Some of these paths have positive phases (constructive interference) and some have negative phases (destructive interference). A PP machine—a theoretical classical computer that non-deterministically guesses answers and only needs to be right slightly more than half the time—is perfectly engineered for this kind of mathematical balancing act.

We can cleverly program the PP machine so that its accepting and rejecting branches physically mirror the positive and negative phase contributions of the quantum circuit [ADH97]. Because the PP class allows for an incredibly razor-thin margin of error, it can absorb the bounded 66% success rate of the BQP machine and translate it into a classical probability that simply floats above 50%. In short, the theoretical power of unbounded classical guessing is vast enough to mathematically simulate quantum wave interference. □

Theorem 5: PP $\subseteq$ PSPACE

Proof. Finally, can we simulate that incredibly powerful PP guessing machine using limited memory? Absolutely, by using the exact same depth-first search trick we applied in Theorem 3.

The probabilistic PP machine generates a massive decision tree of random coin flips. Instead of evaluating the whole tree at once, a deterministic classical machine can methodically walk down one branch at a time. It keeps a running classical counter of how many branches ultimately end in an "accept" state. Once it painstakingly finishes checking every single branch, it simply asks one question: is the final counter greater than half of the total branches?

Keeping track of the current path and maintaining a single numerical counter only takes a polynomial amount of memory space. Therefore, the PP class fits securely inside the bounds of PSPACE. $\square$

Chapter 2

Foundational Quantum Algorithms

2.1 Grover's Algorithm

2.1.1 The Unstructured Search Problem

Let's consider problem of searching an unstructured list or database. Let $X = \{0,1\}^n$ be a search space consisting of $N = 2^n$ distinct elements. We have been given a black box boolean function (commonly referred to as an oracle), denoted by $f : X \to \{0,1\}$. Our goal is to find a specific target element $x^* \in X$ such that $f(x^*) = 1$, while for all other elements $x \neq x^*$, $f(x) = 0$ [NC10].

Using a classic computers with lack of underlying structure or some specific order of elemetns in the database have no algorithmic strategy that can do it faster than just a linear search. To find the proper element, an algorithm must evaluate $f(x)$ for every element in the database which could be up to N times, which at the worst case time will result in complexity of $\mathcal{O}(N)$ and in an average case using $N/2$ queries. Grover's algorithm allows to improve this limit. It allows to finding such target using only $\mathcal{O}(\sqrt{N})$ queries.

2.1.2 Algorithm Structure

The algorithm operates on n qubits initializated to the ground state $|0\rangle^{\otimes n}$. The idea of the procedure relies on the repeated application of an operator $\mathcal{G}$, defined as the Grover iteration.

At the start of the algorithm, the system is put into a uniform superposition $|s\rangle = \frac{1}{\sqrt{n}} \sum_{i=0}^{n-1} |i\rangle$ using the Hadamard transform $H^{\otimes n}$. The Grover iteration $\mathcal{G} = U_s U_f$ consist of two unitary operators [KLM07]:

1. **The Phase Oracle (U_f):** This operator flags the target element by applying a conditional phase shift. It is defined as $U_f = I - 2\,|x^*\rangle\langle x^*|$. It acts as an identity matrix for all states that are not the target state $|x^*\rangle$. Meanwhile it flips the phase of the target state transforming, which also can be noted as $|x\rangle \to (-1)^{f(x)}\,|x\rangle$.

2. **The Diffusion Operator (U_s):** This operator behaves like inversion about the mean amplitude. It is defined as $U_s = 2|s\rangle\langle s| - I$, where $|s\rangle$ is the initial uniform superposition state created at the start of the algorithm. The U_s is constructed by conjugating a global phase shift of the ground state with Hadamard gate, which can be noted as: $U_s = H^{\otimes n}(2|0\rangle\langle 0| - I)H^{\otimes n}$.

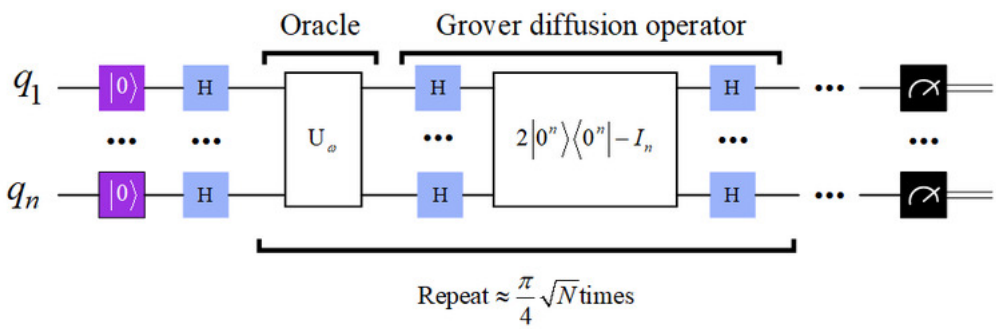

Figure 2.1: General quantum circuit for Grover's algorithm. The initialization phase prepares a uniform superposition, followed by R applications of the Grover iteration (Oracle and Diffuser) [Yua+24].

The whole process is described in more details below in Algorithm 1.

Algorithm 1 Grover's Search Algorithm

1: **Input:** A oracle U_f, eavluating $f(x)$ and flipping the phase of $|x^*\rangle$ (it is our black box).
2: **Output:** The target state $|x^*\rangle$ with success probability $P \approx 1$.
3: Initialize a n qubits to the base state: $|\psi_0\rangle = |0\rangle^{\otimes n}$
4: Apply the Hadamard gate $H^{\otimes n}$ to get a uniform superposition $|s\rangle$:
$|s\rangle = \frac{1}{\sqrt{N}}\sum_{x=0}^{N-1}|x\rangle$
5: Calculate the number of iterations: $R = \lfloor \frac{\pi}{4}\sqrt{N} \rfloor$
6: **for** $i = 1$ to R **do**
7: Apply Oracle U_f: $|x\rangle \to (-1)^{f(x)}|x\rangle$
8: Apply Diffuser $U_s = 2|s\rangle\langle s| - I$: (inversion around the mean amplitude)
9: **end for**
10: Measure the qubits.

2.1.3 Geometric Proof of Correctness

The correctness and idea why Grover's algorithm works are best understood through a geometric lens [Bra+02; Boy+98]. The initial state and all it's subsequent operations

are symmetric with respect to the elements other than a target one. Due to that entire execution of the algorithm can be restricted to a real, two dimensional Hilbert subspace.

We define an orthonormal basis for this space. It will constist of two vectors, the target state $|x^*\rangle$ and a state $|s'\rangle$, where $|s'\rangle$ will be the uniform superposition of all other elements:

$$\left|s'\right\rangle = \frac{1}{\sqrt{N-1}} \sum_{x \neq x^*} |x\rangle \tag{2.1}$$

Using that fact we can now write a starting uniform superposition $|s\rangle$ as a linear combination of these two basis vectors:

$$|s\rangle = \sin(\theta)\,|x^*\rangle + \cos(\theta)\left|s'\right\rangle \tag{2.2}$$

where the initial angle θ is defined by $\sin(\theta) = \frac{1}{\sqrt{N}}$.

Within that 2D plane, the operations U_f and U_s can be interpreted by the following geometric interpretations:

- $U_f = I - 2\,|x^*\rangle\,\langle x^*|$ applies a negative sign only to $|x^*\rangle$. This would be a **reflection across the $|s'\rangle$ axis**.
- $U_s = 2\,|s\rangle\,\langle s| - I$ preserves $|s\rangle$ but flips the sign of any vector orthogonal to it. This would be a **reflection across the $|s\rangle$ vector**.

On a two dementional plane, the product of two reflections results in a rotation. The angle between the reflection axes ($|s'\rangle$ and $|s\rangle$) is θ and their product is $\mathcal{G} = U_s U_f$, which means that the rotation angle is 2θ towards the target state $|x^*\rangle$.

The transformation matrix for a single Grover iteration is:

$$\mathcal{G} = \begin{pmatrix} \cos(2\theta) & \sin(2\theta) \\ -\sin(2\theta) & \cos(2\theta) \end{pmatrix} \tag{2.3}$$

Applying $\mathcal{G}$ k times to the initial state $|s\rangle$ in iteration, just compounds this rotation k times, which can be noted as:

$$\mathcal{G}^k\,|s\rangle = \sin((2k+1)\theta)\,|x^*\rangle + \cos((2k+1)\theta)\left|s'\right\rangle \tag{2.4}$$

The probability of observing the correct target state, which also means the correct output of the algorithm after k iterations is given by the squared amplitude: $P_k = \sin^2((2k+1)\theta)$.

To maximize this probability of the correct output, we want to ensure the state vector is align as closely as possible to the $|x^*\rangle$ axis, that means the argument of the sine function should be close to $\pi/2$:

$$(2k+1)\theta \approx \frac{\pi}{2} \implies k \approx \frac{\pi}{4\theta} - \frac{1}{2} \tag{2.5}$$

Assuming that N is big enough we can use small angle approximation, which means:

$$\theta \approx \sin(\theta) = \frac{1}{\sqrt{N}} \tag{2.6}$$

Which after putting this into our equation gives us the optimal number of Grover iterations:

$$k \approx \frac{\pi}{4}\sqrt{N} \tag{2.7}$$

Or being more precisely:

$$k \approx \lfloor \frac{\pi}{4}\sqrt{\frac{N}{M}} \rfloor \tag{2.8}$$

where M is the number of solutions.

This geometric example exposes a problem of that algorithm. Amplitude amplification is periodic, not monotonic. If the algorithm will be executed more iterations than $k \approx \frac{\pi}{4}\sqrt{\frac{N}{M}}$ times, the state vector can rotate past the $|x^*\rangle$ axis and go back toward the uniform superposition again, which will result in decreasing the probability of success. This behavior is known in the literature as the *overshooting effect* [Boy+98].

The time complexity is heavily dominated by the k oracle evaluations, as the rest uses only the constant number of gates. It is easy to assume that each iteration also consists of a constant number of elementary gates, but here is the catch. Constructing the MCX gate that is used in the diffuser requires $\mathcal{O}(n)$ or $\mathcal{O}(n^2)$ elementary gates (depending on whether ancilla qubits are available) [Bar+95]. Since $n = \log_2 N$, the total number of the complexity is $\mathcal{O}(\sqrt{N}(\log N + T_f))$, where T_f is a number of gates that the U_f is constructed from. However, using the Solovay-Kitaev theorem, it can be constructed with the accuary

of ϵ using only $\mathcal{O}(polylog(\frac{1}{\epsilon}))$. Knowing that, we may clearly tell that Gorver algorithm is in BQP class. It have been proven by Zalka and Bennett et al. [Zal99; Ben+97], that the $\mathcal{O}(\sqrt{N})$ bound is asymptotically optimal for the unstructured search problem.

2.1.4 Synthesis and Implementation

Trying to put the theory of the algorithm to a practical implementation result in necessary of decomposing the abstract operators into elementary hardware instructions. In our case the algorithm was implemented using the Qiskit quantum framework [Qis23].

The design architecture isolates the problem specific Oracle (as it will be given on the input) from the generalized Grover Diffuser (U_s). To create the diffuser expression $2\left|s\right\rangle\left\langle s\right| - I$ it will be required to use a specific sequence of baiscs gates. As shown in code below, the quantum circuit will be constructed as follows:

1. A global layer of Hadamard gates transforms the computational basis into the phase basis, which results in converting the $|s\rangle$ state into a $|0\rangle^{\otimes n}$ state.

2. Pauli-X gate flip the register, mapping the $|0\rangle^{\otimes n}$ state to the $|1\rangle^{\otimes n}$ state so that multi-controlled gates can properly target it.

3. A multi controlled Z gate applies the necessary negative phase shift. In Qiskit, this is synthesized using a multi-controlled X (MCX) gate, bordered by Hadamard gates on the target qubit which shift the bit flip into a phase flip [Bar+95]. As noted earlier, this MCX gate consist of $\mathcal{O}(n)$ or $\mathcal{O}(n^2)$ elementary gates in the physical circuit, where n is the number of qubits.

4. Pauli-X and Hadamard gates applied again to reverse the basis transformations and complete the reflection.

Construction of the Diffusion Operator (U_s) using Qiskit.

```python
def _build_diffuser(self) -> QuantumCircuit:
    qc = QuantumCircuit(self.num_qubits, name="Diffuser")

    # 1. Transform back to the computational basis
    qc.h(range(self.num_qubits))

    # 2. Invert states to target the |0...0> component
    qc.x(range(self.num_qubits))

    # 3. Apply Multi-Controlled Z to flip the phase
    target_qubit = self.num_qubits - 1
    control_qubits = list(range(self.num_qubits - 1))

    qc.h(target_qubit)
    if control_qubits:
        qc.mcx(control_qubits, target_qubit)
    qc.h(target_qubit)

    # 4. Revert the state inversion and basis change
    qc.x(range(self.num_qubits))
    qc.h(range(self.num_qubits))

    return qc
```

2.2 Advanced Variants of Grover's Algorithm

As mentioned in the previous section, the standard Grover's algorithm is optimal for general unstructured search, but it has a problem in it. It's success probability is not exactly 100%, as also, what will be a bigger issue, it has the overshooting effect. To address these issues several other variants of the Grover's algorithm have been developed.

2.2.1 Exact Quantum Search

Idea of Exact Quantum Search

In the standard version of Grover's algorithm we perform $k \approx \frac{\pi}{4}\sqrt{\frac{N}{M}}$ iterations, where M is the number of solutions. Let's further denote $\sqrt{\frac{N}{M}}$ as Q As we know k must be an integer which mean it already an apporximation so we may not align perfectly with the correct angle. Also due to the formula using small angle approximation, the number of iterations may differ even more from the target state vector $|x^*\rangle$. The final success probability $P_k = \sin^2((2k+1)\theta)$ is typically quite close to 1, but sometimes that may result in an issue.

The *Exact Quantum Search* variant, first proposed by Long [Lon01], modifies the algorithm to guarantee correct answer (excluding the errors from floating point arithmetic). In short, it peforms the standard Grover iterations until the state vector is within a single step of the target, then using one step it will try to align with the target angle perfectly.

In the standard algorithm, every iteration rotates the state vector by an angle of 2θ. After k full iterations, the total angle relative to the starting uniform superposition is $(2k+1)\theta$. As described earlier, to perform a correct measurement, that angle should equal to $\frac{\pi}{2}$. So to solve the problem with k being a whole number, we should at the end rotate the vector by δ, which is defined as $\delta = \frac{\pi}{2} - (2k+1)\theta$.

To also ensure that this computation is fully accurate, we must stop relying on the small angle approximation (which was $\theta \approx \frac{1}{Q}$). Instead we will calculate the initial angle using a formula:

$$\theta = \arcsin\left(\frac{1}{Q}\right) \tag{2.9}$$

Due to that the gap δ is known precisely.

The inital angle formula comes from the definition of the uniform superposition. At the start of the algorithm, the probability amplitude of finding the target state is exactly $\frac{1}{Q}$ (comes right from the distribution after using the Hadamard gate). In the geometric representation, this initial amplitude corresponds to the sine of the starting angle θ, which gives us the exact equation $\sin(\theta) = \frac{1}{Q}$. By applying the inverse sine function, we can extract the precise angle.

Using that two improvements we can in theory achieve the deterministic result of the search algorithm.

Implementation of Exact Quantum Search

To implement this variant we must implement custom parameterized diffusion operator to apply specific fractional phase shifts, which in that case will be the exact remaining angle.

The code below shows the implementation of the Exact Quantum Search. The algorithm performs standard iterations first. Then it calculates the remaining angle and applies generated diffuser for the final fractional step.

Implementation of the Exact Quantum Search algorithm.

```python
class GroverAlgorithmExact(GroverAlgorithmBase):
    def __init__(self, oracle):
        super().__init__(oracle)
        self.standard_diffuser = self._build_diffuser(np.pi)

    def _build_diffuser(self, angle: float):
        qc = QuantumCircuit(self.num_qubits, name=f"U_s")
        qc.h(range(self.num_qubits))
        qc.x(range(self.num_qubits))

        if self.num_qubits > 1:
            mcphase = MCPhaseGate(angle, self.num_qubits - 1)
            qc.append(mcphase, range(self.num_qubits))
        else:
            qc.p(angle, 0)

        qc.x(range(self.num_qubits))
        qc.h(range(self.num_qubits))
        return qc

    def build_circuit(self, num_solutions: int = None):
        if number_of_solutions is None or number_of_solutions < 1:
            raise ValueError("Invalid number of solutions.")

        n_states = 2 ** self.num_qubits
        theta = 2 * np.arcsin(np.sqrt(num_solutions / n_states))
        exact_iters_float = (np.pi / (2 * theta)) - 0.5
        iterations = max(0, math.floor(exact_iters_float))

        qc = QuantumCircuit(self.num_qubits, self.num_qubits)
        qc.h(range(self.num_qubits))

        for _ in range(iterations):
            qc.compose(self.oracle, inplace=True)
            qc.compose(self.standard_diffuser, inplace=True)

        # Perform the final fractional iteration
        remaining_angle = np.pi/2 - (iterations + 0.5) * theta
        if remaining_angle > 1e-5:
            final_phase = 2 * remaining_angle
            final_diffuser = self._build_diffuser(final_phase)
            qc.compose(self.oracle, inplace=True)
            qc.compose(final_diffuser, inplace=True)

        qc.measure(range(self.num_qubits), range(self.num_qubits))
        return qc
```

2.2.2 Fixed-Point Quantum Search

Idea of Fixed-Point Quantum Search

A significant problem of the standard algorithm is its periodic. If the number of target solutions M is unknown, it is impossible (or at least hard) to compute the optimal number of iterations k to perform. Performing too may iterations in the algorithm causes the state vector to rotate past the $|x^*\rangle$ axis, which results in decreasing the success probability (Overshooting effect).

The *Fixed-Point Quantum Search* addresses this by changing the geometry of the operations. Introduced by Yoder, Low, and Chuang [YLC14], this variant replaces the standard π (180°) phase shifts in both the phase oracle (U_f) and the diffusion operator (U_s) with a phase shift of $\pi/3$ (60°).

Proof of Convergence

To prove that replacing the standard phase shifts to $\pi/3$ will not be periodic but convergent, we must analyze failure probability amplitude and prove that it is strictly decreasing. Let the initial uniform superposition state be defined as $|s\rangle = \sin\gamma\,|x^*\rangle + \cos\gamma\,|s'\rangle$, where $|s'\rangle$ is the uniform superposition of all states different than a target one and γ is the initial angle.

As said before in this variant we will use phase shift of $\omega = e^{i\pi/3}$. It follows that the modified oracle $U_f(\pi/3)$ and diffuser $U_s(\pi/3)$ operators can be denoted as:

$$U_f = I - (1-\omega)\,|x^*\rangle\,\langle x^*| \tag{2.10}$$

$$U_s = I - (1-\omega)\,|s\rangle\,\langle s| \tag{2.11}$$

Applying the oracle to the initial state yields $U_f\,|s\rangle = \omega\sin\gamma\,|x^*\rangle + \cos\gamma\,|s'\rangle$. Also applying a one full new iteration gives us the following state:

$$\mathcal{G}_{\pi/3}\,|s\rangle = U_s U_f\,|s\rangle = (I - (1-\omega)\,|s\rangle\,\langle s|)(U_f\,|s\rangle) = U_f\,|s\rangle - (1-\omega)\,|s\rangle\,\langle s|\,U_f\,|s\rangle \tag{2.12}$$

Let's observe the amplitude of the state different than the target one. This amplitude will represent the error of our algorithm and can be denoted as $\langle s'|\,U_s U_f\,|s\rangle$:

$$\langle s'|\,U_s U_f\,|s\rangle = \langle s'|\,U_f\,|s\rangle - (1-\omega)\,\langle s'|s\rangle\,\langle s|\,U_f\,|s\rangle \tag{2.13}$$

Using a few substitutions form the definition of $|s\rangle$ and $U_f\,|s\rangle$ we can evaluate a few terms:

$$\langle s'|s\rangle = \sin\gamma\,\langle s'|x^*\rangle + \cos\gamma\,\langle s'|s'\rangle = \sin\gamma\cdot 0 + \cos\gamma\cdot 1 = \cos\gamma \tag{2.14}$$

$$\langle s'|\,U_f\,|s\rangle = \omega\sin\gamma\,\langle s'|x^*\rangle + \cos\gamma\,\langle s'|s'\rangle = \omega\sin\gamma\cdot 0 + \cos\gamma\cdot 1 = \cos\gamma \tag{2.15}$$

$$\langle s|\,U_f\,|s\rangle = (\sin\gamma\,\langle x^*| + \cos\gamma\,\langle s'|)(\omega\sin\gamma\,|x^*\rangle + \cos\gamma\,|s'\rangle) = \omega\sin^2\gamma + \cos^2\gamma \tag{2.16}$$

Using all that we obtain that our total error can be represented as:

$$\langle s'|\,U_sU_f\,|s\rangle = \cos\gamma - (1-\omega)\cos\gamma(\omega\sin^2\gamma + \cos^2\gamma) \tag{2.17}$$

Factoring out $\cos\gamma$ simplifies the expression to:

$$\cos\gamma[1 - (\omega-\omega^2)\sin^2\gamma - (1-\omega)\cos^2\gamma] \tag{2.18}$$

We know that $\omega = e^{i\pi/3}$ is a root of the equation $\omega^2 - \omega + 1 = 0$, which implies that $\omega - \omega^2 = 1$. Substituting this into the above equation gives us:

$$\cos\gamma[1 - \sin^2\gamma - (1-\omega)\cos^2\gamma] = \cos\gamma[\cos^2\gamma - (1-\omega)\cos^2\gamma] = \omega\cos^3\gamma \tag{2.19}$$

The value of amplitude for non target elements after one iteration equals to $\cos^3\gamma$. As we know that the initial angle $\gamma > 0$ (as the at least one target element exists), we know that $\cos\gamma < 1$.

After k iterations, the error amplitude can be bounded by $\cos^{2k+1}\gamma$, which now can be clearly seen approaches 0 as $k \to \infty$. That means that the probability of measuring the correct state approaches 1 and the overshooting effect would not have place there.

Implementation of Fixed-Point Quantum Search

The implementation is quite similar to the oryginal Grover implementation. The main difference here is to create oracle and diffuser $U_s(\pi/3)$, which will be applyed as in the classic Grover's algorithm. The problem here in the general will be that $U_f(\pi/3)$ cannot be created without a knowledge about the black box and has to be given as an input. In practice Qiskit can handle that as it knows how that oracle is constructed but it may not always be such easy case.

Implementation of the Fixed-Point Quantum Search algorithm.

```python
class GroverAlgorithmFixedPoint(GroverAlgorithmBase):
    def __init__(self, oracle):
        super().__init__(oracle)

        self.diffuser_pi_3 = self._build_diffuser(np.pi / 3)
        self.oracle_pi_3 = self.oracle.power(1/3)

    def _build_diffuser(self, angle: float) -> QuantumCircuit:
        qc = QuantumCircuit(self.num_qubits, name=f"U_s")
        qc.h(range(self.num_qubits))
        qc.x(range(self.num_qubits))

        if self.num_qubits > 1:
            mcphase = MCPhaseGate(angle, self.num_qubits - 1)
            qc.append(mcphase, range(self.num_qubits))
        else:
            qc.p(angle, 0)

        qc.x(range(self.num_qubits))
        qc.h(range(self.num_qubits))
        return qc

    def build_circuit(self,  num_solutions: int = None):
        opt_its = self._calculate_optimal_iterations(num_solutions)

        # It will converge securely without overshooting
        iterations = 2*(opt_its) + 17

        qc = QuantumCircuit(self.num_qubits, self.num_qubits)
        qc.h(range(self.num_qubits))

        # Iterations converge securely without overshooting
        for _ in range(iterations):
            qc.compose(self.oracle_pi_3, inplace=True)
            qc.compose(self.diffuser_pi_3, inplace=True)

        qc.measure(range(self.num_qubits), range(self.num_qubits))
        return qc
```

Bibliography

[EPR35] Albert Einstein, Boris Podolsky, and Nathan Rosen. "Can quantum-mechanical description of physical reality be considered complete?" In: *Physical review* 47.10 (1935), p. 777.

[Dir39] Paul Adrien Maurice Dirac. "A new notation for quantum mechanics". In: *Mathematical Proceedings of the Cambridge Philosophical Society* 35.3 (1939), pp. 416–418.

[Bel64] John S Bell. "On the Einstein Podolsky Rosen paradox". In: *Physics Physique Fizika* 1.3 (1964), p. 195.

[Ben73] Charles H Bennett. "Logical reversibility of computation". In: *IBM journal of Research and Development* 17.6 (1973), pp. 525–532.

[Ben80] Paul Benioff. "The computer as a physical system: A microscopic quantum mechanical Hamiltonian model of computers as represented by Turing machines". In: *Journal of statistical physics* 22.5 (1980), pp. 563–591.

[Fey82] Richard P Feynman. "Simulating physics with computers". In: *International journal of theoretical physics* 21.6-7 (1982), pp. 467–488.

[Deu85] David Deutsch. "Quantum theory, the Church-Turing principle and the universal quantum computer". In: *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 400.1818 (1985), pp. 97–117.

[Deu89] David Deutsch. "Quantum computational networks". In: *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 425.1868 (1989), pp. 73–90.

[Yao93] Andrew Chi-Chih Yao. "Quantum circuit complexity". In: *Proceedings of 1993 IEEE 34th Annual Foundations of Computer Science*. IEEE. 1993, pp. 352–361.

[Sho94] Peter W Shor. "Algorithms for quantum computation: discrete logarithms and factoring". In: *Proceedings 35th annual symposium on foundations of computer science*. Ieee. 1994, pp. 124–134.

[Bar+95] Adriano Barenco et al. "Elementary gates for quantum computation". In: *Physical review A* 52.4 (1995), p. 3457.

[ADH97] Leonard M Adleman, Jonathan DeMarrais, and Ming-Deh Huang. "Quantum computability". In: *SIAM Journal on Computing* 26.5 (1997), pp. 1524–1540.

[Ben+97] Charles H Bennett et al. "Strengths and weaknesses of quantum computing". In: *SIAM journal on Computing* 26.5 (1997), pp. 1510–1523.

[BV97] Ethan Bernstein and Umesh Vazirani. "Quantum complexity theory". In: *SIAM Journal on computing* 26.5 (1997), pp. 1411–1473.

[Boy+98] Michel Boyer et al. "Tight bounds on quantum searching". In: *Fortschritte der Physik: Progress of Physics* 46.4-5 (1998), pp. 493–505.

[Zal99] Christof Zalka. "Grover's quantum searching algorithm is optimal". In: *Physical Review A* 60.4 (1999), p. 2746.

[Lon01] Gui-Lu Long. "Grover algorithm with zero theoretical failure rate". In: *Physical Review A* 64.2 (2001), p. 022307.

[Bra+02] Gilles Brassard et al. "Quantum amplitude amplification and estimation". In: *Contemporary Mathematics* 305 (2002), pp. 53–74.

[DN05] Christopher M Dawson and Michael A Nielsen. "The Solovay-Kitaev algorithm". In: *arXiv preprint quant-ph/0505030* (2005).

[KLM07] Phillip Kaye, Raymond Laflamme, and Michele Mosca. *An introduction to quantum computing*. Oxford University Press, 2007.

[Mer07] N David Mermin. *Quantum computer science: an introduction*. Cambridge University Press, 2007.

[GLM08] Vittorio Giovannetti, Seth Lloyd, and Lorenzo Maccone. "Quantum random access memory". In: *Physical review letters* 100.16 (2008), p. 160501.

[AB09] Sanjeev Arora and Boaz Barak. *Computational complexity: a modern approach*. Cambridge University Press, 2009.

[HHL09] Aram W Harrow, Avinatan Hassidim, and Seth Lloyd. "Quantum algorithm for linear systems of equations". In: *Physical review letters* 103.15 (2009), p. 150502.

[NC10] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge University Press, 2010.

[YLC14] Theodore J Yoder, Guang Hao Low, and Isaac L Chuang. "Fixed-point quantum search with an optimal number of queries". In: *Physical review letters* 113.21 (2014), p. 210501.

[Qis23] Qiskit contributors. *Qiskit: An Open-source Framework for Quantum Computing*. 2023. DOI: `10.5281/zenodo.2562110`. URL: `https://qiskit.org/`.

[Yua+24] Gongsheng Yuan et al. "Quantum Computing for Databases: Overview and Challenges". In: (May 2024). DOI: `10.48550/arXiv.2405.12511`.